История развития ООП. Базовые понятия ООП: объект, его свойства и методы, класс, интерфейс.

«Объектно-ориентированная версия "спагетти-кода" — это, конечно, "код-лазанья". В нем так много слоев, что вы теряете след того, где вообще что-то происходит».

Линус Торвальдс

Что это

Объектно-ориентированное программирование (сокращённо ООП) — это парадигма разработки программного обеспечения, согласно которой приложения состоят из объектов.

На объектах и классах строится всё ООП.

История

История объектно-ориентированного программирования (ООП) — это эволюционный путь борьбы со сложностью кода, который превратил программирование из написания инструкций для процессора в моделирование объектов реального мира.

1950-е — начало 1960-х

В эпоху зарождения ИТ программы писались на ассемблере и ранних процедурных языках (Fortran, Algol). Пока программы были небольшими, этот подход работал. Однако с ростом масштабов систем кодовые базы превращались в «спагетти-код» — изменения в одной части программы непредсказуемо ломали другие её части, так как данные и функции существовали отдельно друг от друга. Возникла острая необходимость объединить данные с методами их обработки

1950-е — начало 1960-х

В эпоху зарождения ИТ программы писались на ассемблере и ранних процедурных языках (Fortran, Algol). Пока программы были небольшими, этот подход работал. Однако с ростом масштабов систем кодовые базы превращались в «спагетти-код» — изменения в одной части программы непредсказуемо ломали другие её части, так как данные и функции существовали отдельно друг от друга. Возникла острая необходимость объединить данные с методами их обработки



Simula (1960-e)

Первым языком программирования, в котором официально появились классы, объекты, наследование и динамическое связывание, стал Simula 67, разработанный норвежскими учеными Оле-Йоханом Далем и Кристеном Нюгордом.

https://github.com/AKT-Polina/OAiP/blob/main/Lections/simula.sim

Algol 68

в нем появились механизмы, отдаленно напоминающие составные части ООП:

- Структуры данных (struct): Позволяли объединять разнородные данные (аналог полей/атрибутов класса).
- Ссылки (ref): Давали возможность строить сложные динамические структуры и связывать данные.
- Процедурные переменные позволяли передавать логику как данные, что отдаленно напоминало таблицы виртуальных методов.

Классы и структуры

Критерий	Структуры (`struct`)	Классы (`class`)
Тип данных	Обычно **значимый тип** (Value Type). Копируется при передаче.	**Ссылочный тип** (Reference Type). Передается по ссылке.
Размещение в памяти	В **стеке** (Stack). Быстрое выделение и очистка памяти.	В **куче** (Heap). Памятью управляет сборщик мусора или счетчик ссылок.
Наследование	Обычно **не поддерживают** наследование от других структур.	**Поддерживают** полноценное многоуровневое наследование.
В C++ (специфика)	Доступ к членам по умолчанию `public` .	Доступ к членам по умолчанию `private` .
Размер и сложность	Предназначены для **легковесных** и простых объектов.	Предназначены для **сложных систем** со своей логикой.

Класс

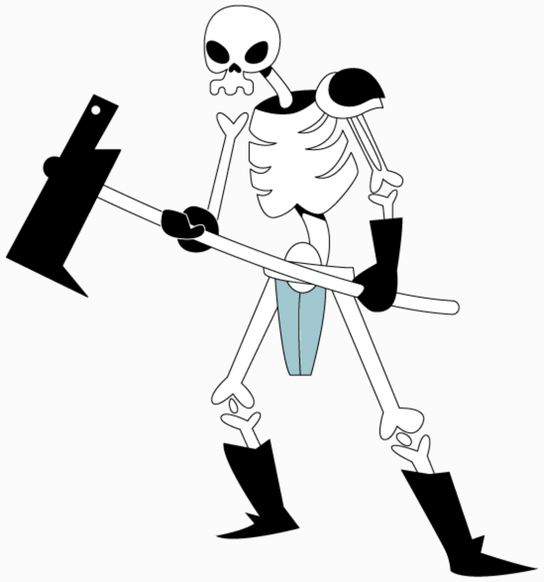
Класс — это тип данных, созданный пользователем. Он содержит разные свойства и методы, как, например, тип `String` или `Int`.

Объект

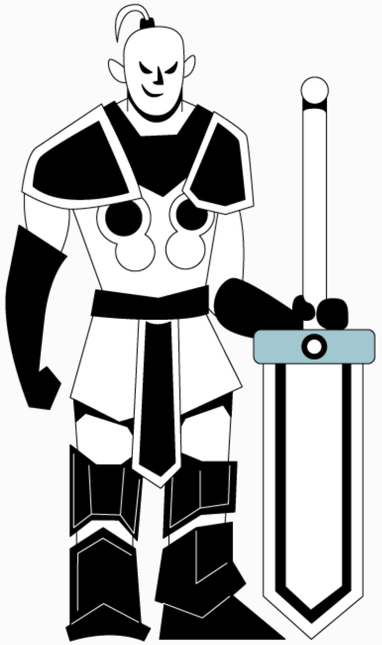
Объект — это экземпляр класса, или его копия, которая находится в памяти компьютера. Например, когда вы создаёте переменную типа `String` и присваиваете ей значение «Строка», то в памяти создаётся экземпляр класса `String`.

По-другому можно сказать, что объекты — это сущности, у которых есть свойства и поведение. Обычно объекты являются экземплярами какого-нибудь класса.

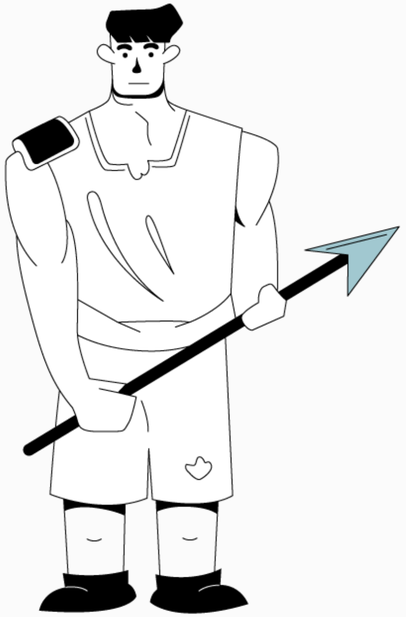
Character



Hero



NPC

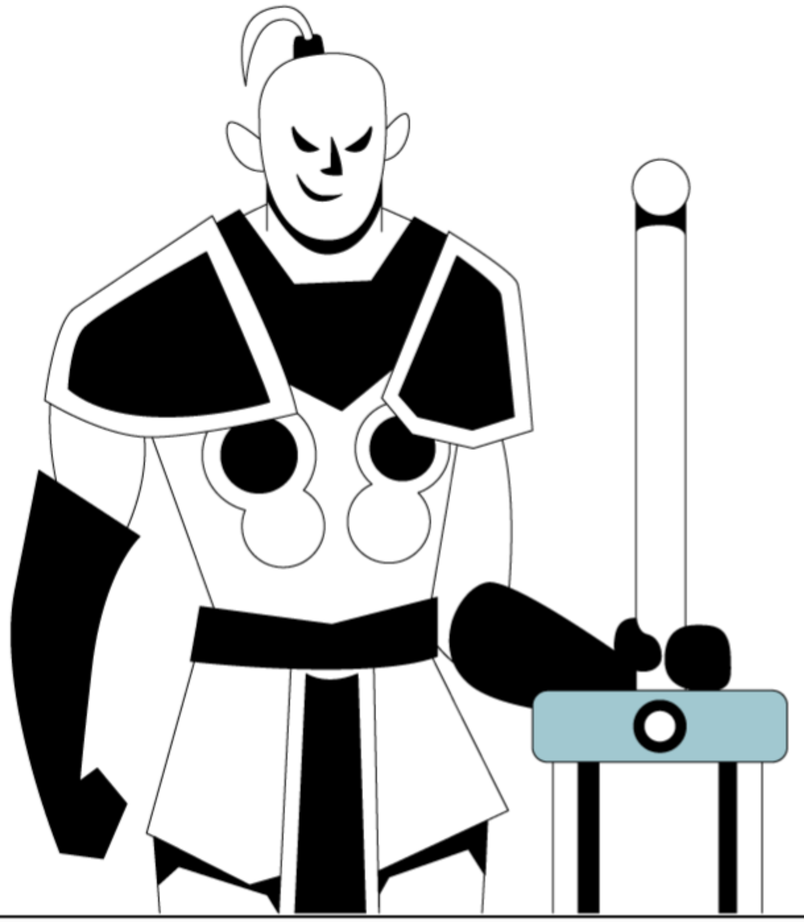


Свойства

Свойства — это данные, которые связаны с конкретным объектом:

- здоровье,
- очки,
- деньги,
- сила,
- ловкость,
- интеллект,
- скорость,
- координаты.

Hero



Здоровье — 100

Очки — 100

Деньги — 100

Сила — 5

Ловкость — 5

Интеллект — 4

Скорость — 100

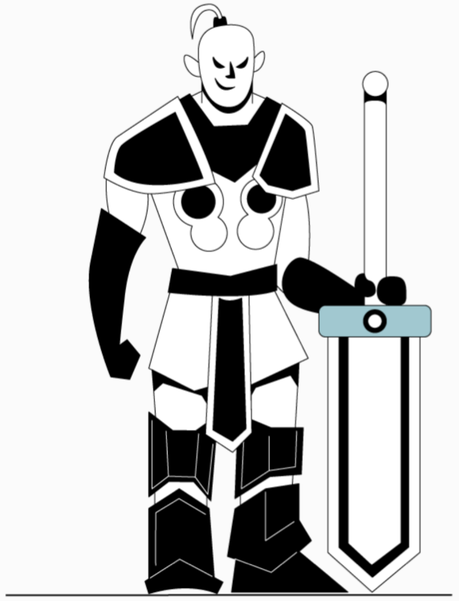
Координаты — 1,4

Методы

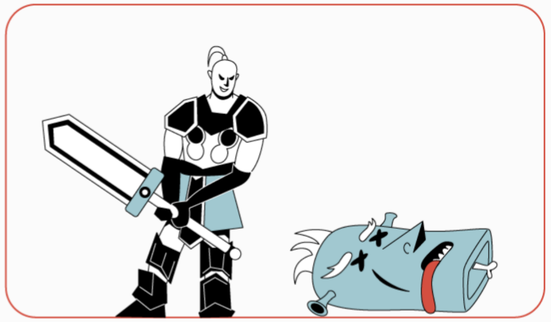
Поведение объекта определяется с помощью методов — специальных блоков кода, которые можно вызывать из разных частей программы. Например, у того же объекта `Character` могут быть следующие методы:

- идти,
- атаковать,
- говорить,
- подобрать,
- выбросить,
- использовать.

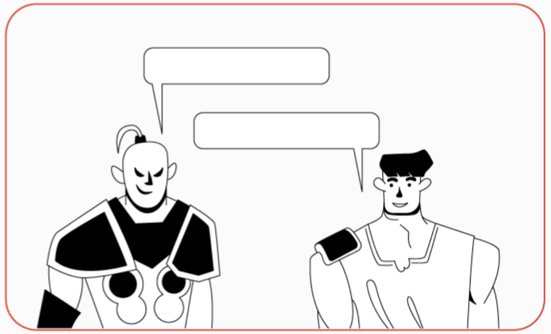
Hero



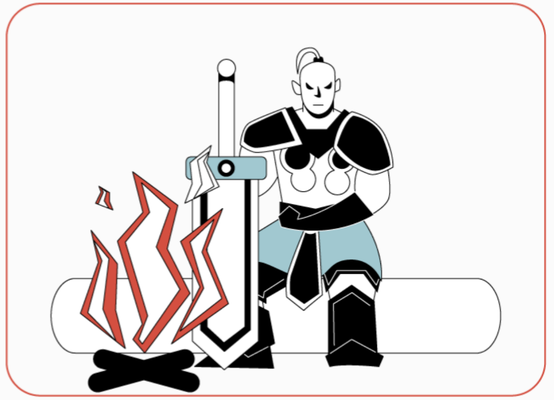
Атаковать



Говорить



Отдыхать



Для чего это все

Используя эти свойства и методы, можно значительно ускорить разработку, сделать код более читаемым. К тому же самому программисту проще составлять код, если он думает с помощью объектов.

Разработчики не пишут какую-то функцию, которая будет делать что-то для программы в целом. Вместо этого они мысленно разделяют приложение на отдельные компоненты и продумывают их свойства и поведение.

Где это работает

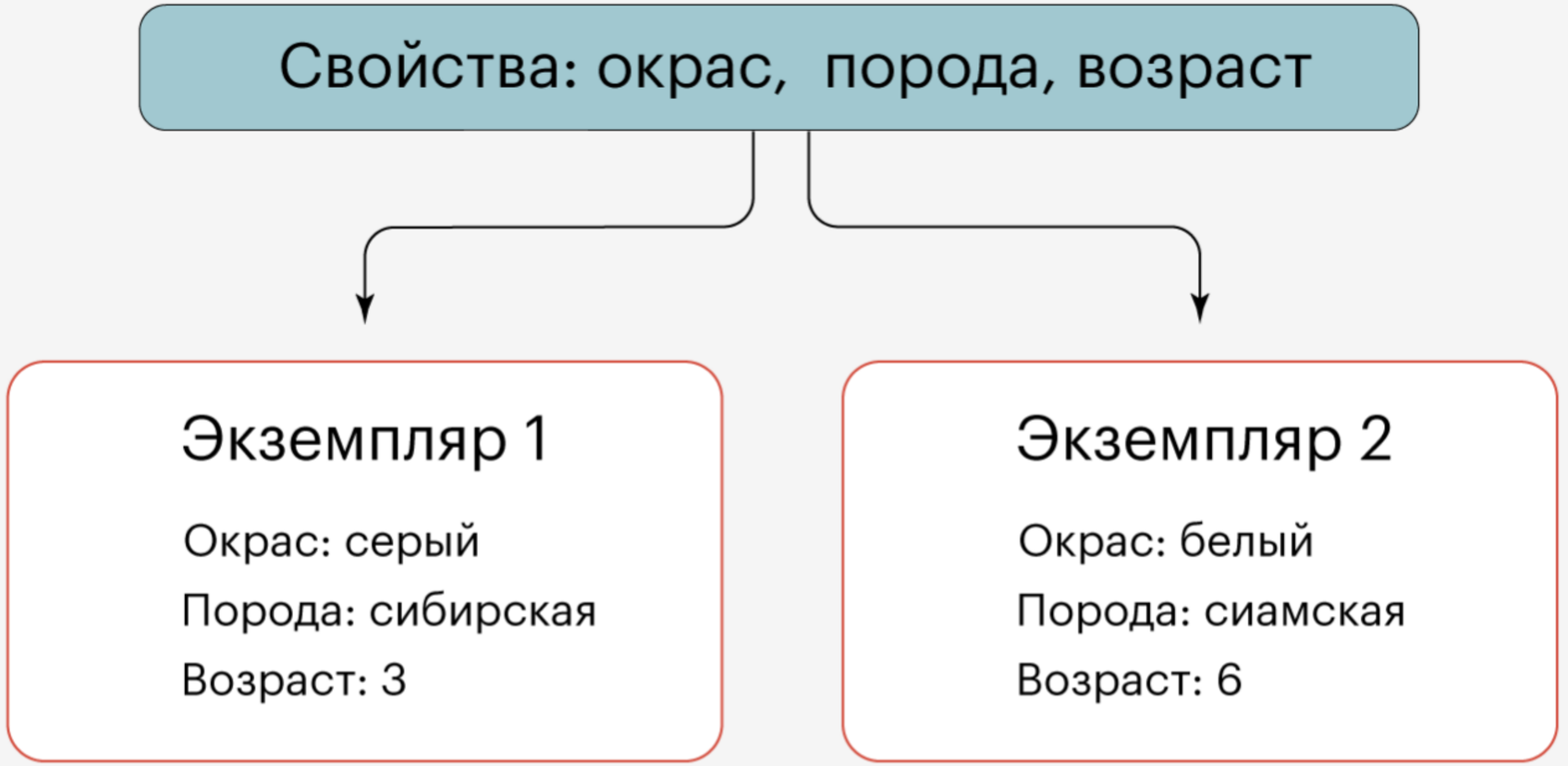
- C#,
- Java,
- Python,
- JavaScript,
- PHP,
- Kotlin,
- Swift,
- Objective-C,
- C++.

Плюсы	Минусы
Легко читается. Не нужно выискивать в коде функции и выяснять, за что они отвечают	**Потребляет больше памяти.** Объекты потребляют больше оперативной памяти, чем примитивные типы данных
Быстро пишется. Можно быстро создать сущности, с которыми должна работать программа	**Снижает производительность.** Многие вещи технически реализованы иначе, поэтому они используют больше ресурсов
Проще реализовать большой набор функций. Так как на написание кода уходит меньше времени, можно гораздо быстрее создать приложение с множеством возможностей	**Сложно начать.** Парадигма ООП сложнее функционального программирования, поэтому на старт уходит больше времени
Меньше повторений. Не нужно писать однотипные функции для разных сущностей	

Основные принципы ООП

- абстракции,
- инкапсуляции,
- наследовании,
- полиморфизме.

Класс «Кошка»


Свойства: окрас, порода, возраст
Экземпляр 1
Окрас: серый
Порода: сибирская
Возраст: 3
Экземпляр 2
Окрас: белый
Порода: сиамская
Возраст: 6

Абстракция

При создании класса мы упрощаем его до тех атрибутов и методов, которые нужны в этом конкретном коде, не пытаясь описать его целиком и отбрасывая всё второстепенное. Скажем, все кошки теоретически умеют охотиться, но если наша программа не предназначена для ловли мышей, то и прописывать этот метод незачем.

Класс «Домашняя кошка»



Свойства:

— окрас
— порода
— возраст

Методы:

— лежать
— просить еду
— бегать по квартире
в три часа ночи

А что ещё нужно домашним кошкам?

Инкапсуляция

Доступ к данным объекта должен контролироваться, чтобы пользователь не мог изменить их в произвольном порядке и что-то поломать. Поэтому для работы с данными программисты пишут публичные методы, которые составляют интерфейс объекта.

Класс «Кошка»

Свойства:

— окрас
— порода
— возраст

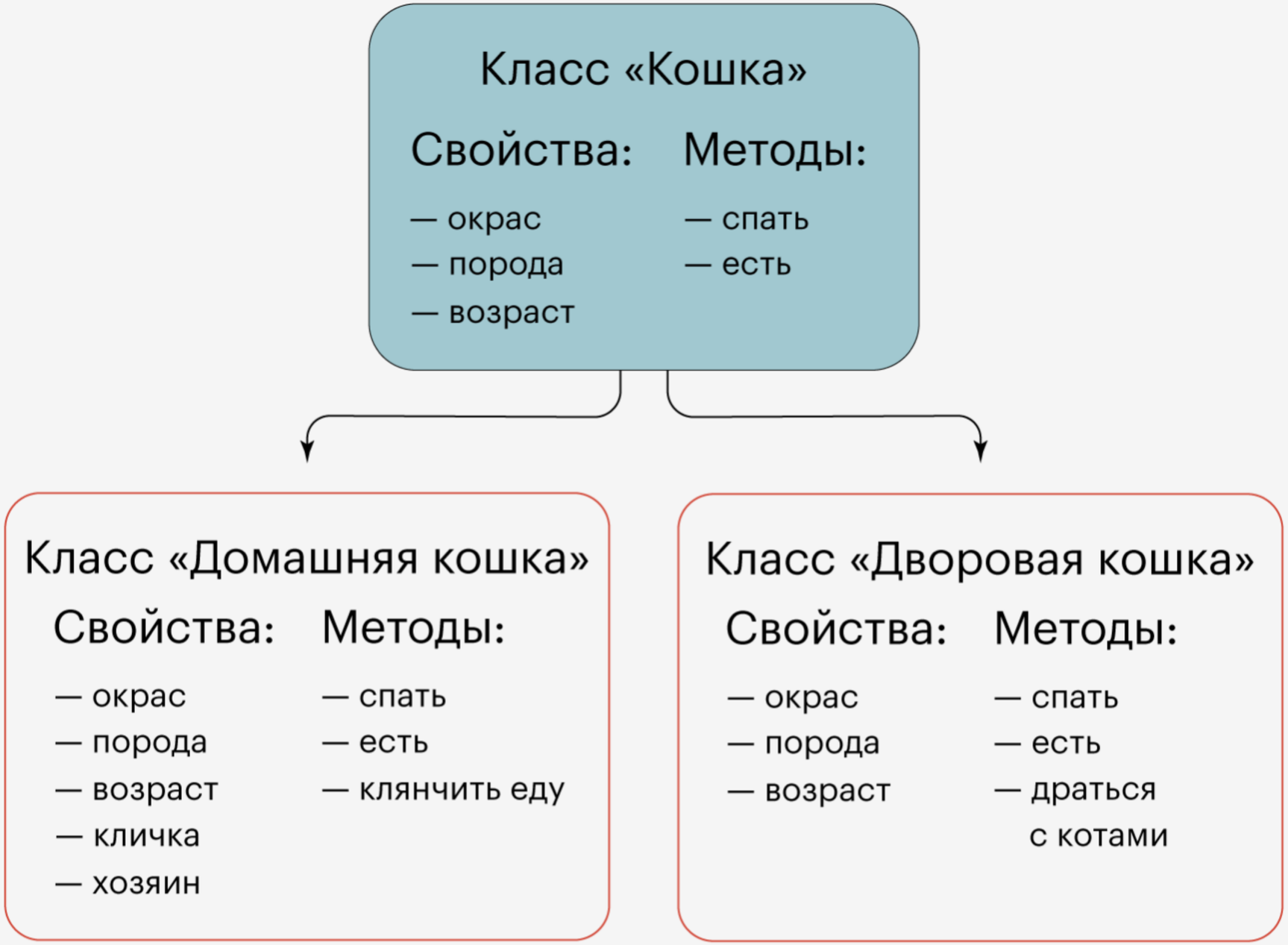
Можно изменить породу моей кошки?

Нет! Можно только посмотреть.



Наследование

Классы могут передавать свои атрибуты и методы классам-потомкам. Например, мы хотим создать новый класс «Домашняя кошка». Он практически идентичен классу «Кошка», но у него появляются новые атрибуты — «хозяин» и «кличка», а также метод «клянчить вкусняшку». Достаточно объявить «Домашнюю кошку» наследником «Кошки» и прописать новые атрибуты и методы — вся остальная функциональность перейдёт от родителя к потомку.


Класс «Кошка»
Свойства:
— окрас
— порода
— возраст
Методы:
— спать
— есть
Класс «Домашняя кошка»
Свойства:
— окрас
— порода
— возраст
— кличка
— хозяин
Методы:
— спать
— есть
— клянчить еду
Класс «Дворовая кошка»
Свойства:
— окрас
— порода
— возраст
Методы:
— спать
— есть
— драться с котами

Полиморфизм

Этот принцип позволяет применять одни и те же команды к объектам разных классов, даже если они выполняются по-разному.

Класс «Кошка»

Свойства:

— окрас
— порода
— возраст

Методы:

— спать
— есть

Класс «Попугай»

Свойства:

— окрас
— порода
— возраст

Методы:

— спать
— есть

Спать



